

CO5 ASSESSMENTS 2: PSEUDOCODE WRITING TASK

Real-Time Computer Vision Pipeline & Algorithm Specification Solutions

Task 1: Real-Time Face Detection and Recognition Pipeline

Constraints Met

Real-Time Optimized

Multi-Face Support

System Architecture: Captures video frames continuously, resizes for minimal delay, detects multiple faces via Haar Cascade / DNN, extracts features, predicts dynamic matches using LBPH/Embeddings, and renders interactive bounding boxes.

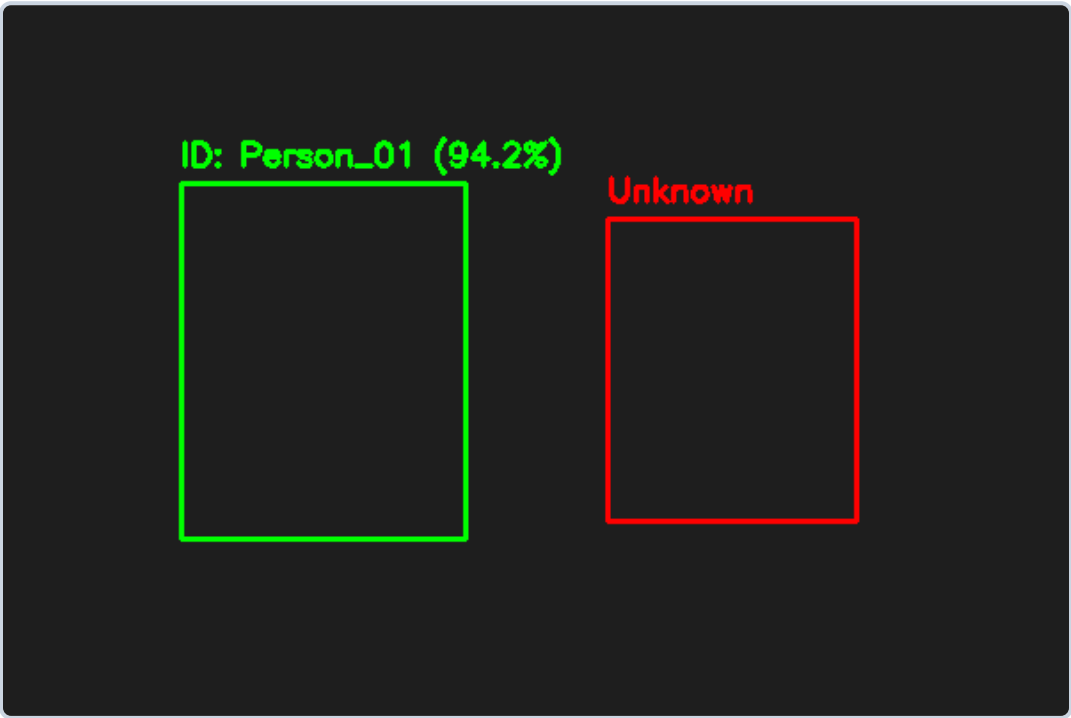


Figure 1.1: Visual output representation of real-time multi-face detection & recognition pipeline.

```
BEGIN RealTime_Face_Recognition_Pipeline

// Step 1: Initialize System Hardware & Model Files
INITIALIZE camera_stream = OpenVideoCamera(device_index = 0)
LOAD face_detector = LoadHaarCascade("haarcascade_frontalface_default.xml")
LOAD face_recognizer = LoadLBPHFaceRecognizer()
LOAD_TRAINED_MODEL(face_recognizer, "trained_face_model.xml")
LOAD database_labels = {1: "Person_01", 2: "Person_02"}
SET CONFIDENCE_THRESHOLD = 65.0

IF NOT camera_stream.isOpened() THEN
    PRINT "Error: Camera stream unavailable."
    EXIT
END IF

// Step 2: Continuous Real-Time Frame Processing Loop
WHILE camera_stream.isOpened() DO

    status, frame = camera_stream.read()
    IF NOT status THEN
        BREAK // Frame read failure or stream disconnect
    END IF

    // Preprocessing for Latency Reduction
    resized_frame = ResizeImage(frame, width = 640) // Standardize image scale
    gray_frame = ConvertToGrayscale(resized_frame)
    equalized_frame = EqualizeHistogram(gray_frame) // Normalize lighting
```

```

// Step 3: Multi-Face Detection
faces_list = face_detector.detectMultiScale(
    equalized_frame,
    scaleFactor = 1.1,
    minNeighbors = 5,
    minSize = (30, 30)
)

// Step 4: Feature Extraction and Recognition per Detected Face
FOR EACH (x, y, width, height) IN faces_list DO

    face_roi = equalized_frame[y : y + height, x : x + width]
    normalized_face = ResizeImage(face_roi, target_size = (100, 100))

    // Predict identity and confidence distance score
    label_id, distance = face_recognizer.predict(normalized_face)

    // Step 5: Identification Logic & Annotation
    IF distance < CONFIDENCE_THRESHOLD THEN
        person_name = database_labels[label_id]
        box_color = COLOR_GREEN // Authorized Person
    ELSE
        person_name = "Unknown"
        box_color = COLOR_RED // Unregistered Identity
    END IF

    display_text = person_name + " (" + ToString(100 - distance) + "%)"

    DrawRectangle(resized_frame, (x, y), (x + width, y + height), box_color, 2)
    DrawText(resized_frame, display_text, (x, y - 10), FONT_HERSHEY, 0.5, box_color)

END FOR

// Step 6: Render Frame & Efficient Key-Break Termination
DisplayWindow("Real-Time Face Recognition", resized_frame)

key_pressed = WaitKey(delay = 1) // Non-blocking 1ms check
IF key_pressed == ASCII_ESC OR key_pressed == 'q' THEN
    BREAK
END IF

END WHILE

// Step 7: System Cleanup
camera_stream.release()
DestroyAllWindows()

END RealTime_Face_Recognition_Pipeline

```

Task 2: Vehicle Detection, Tracking and Counting System

Double-Counting Safeguard

Centroid Tracking

Virtual ROI Line

System Architecture: Employs Background Subtraction (MOG2) with morphological filtering, Euclidean distance centroid tracking across frames, and unique tracking ID set management to eliminate double-counting.

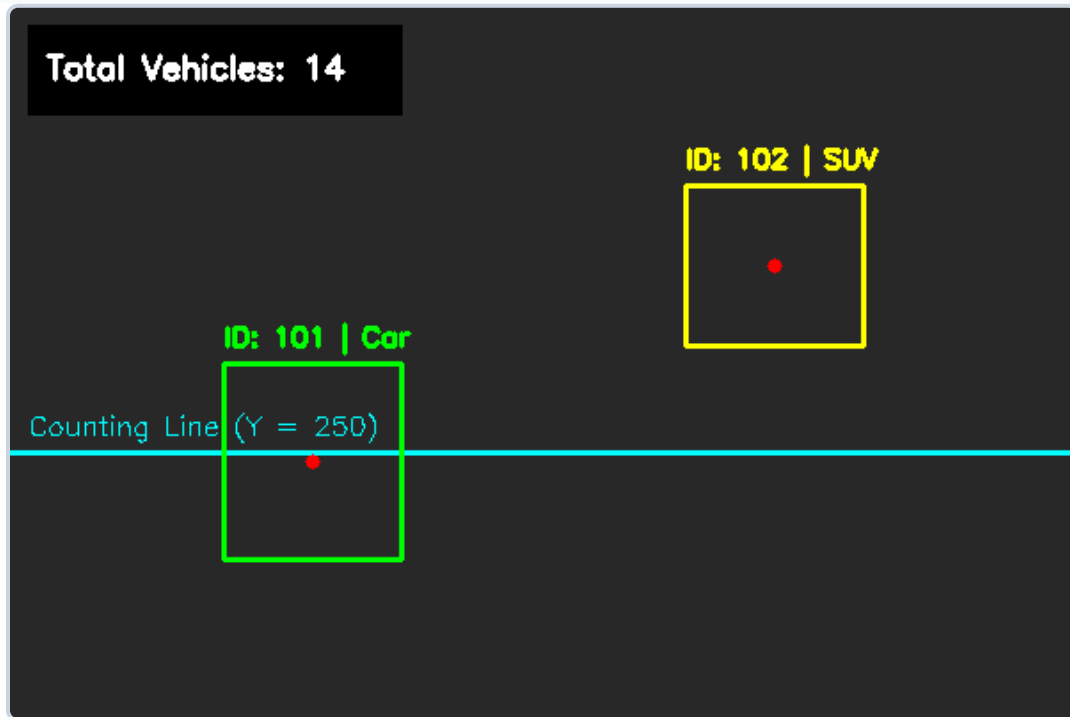


Figure 2.1: Visual output representation of vehicle tracking, ROI virtual line crossing, and counter telemetry.

BEGIN Vehicle_Detection_Tracking_Counting

```
// Step 1: System & Parameter Initialization
INITIALIZE video_stream = OpenVideoFileOrCamera("traffic_feed.mp4")
INITIALIZE bg_subtractor = CreateBackgroundSubtractorMOG2(history=500, varThreshold=50)
INITIALIZE tracker = CreateEuclideanCentroidTracker(maxDisappeared = 10)

SET LINE_Y_COORDINATE = 250
SET LINE_TOLERANCE = 6
SET total_vehicle_count = 0
INITIALIZE counted_ids_set = EMPTY_SET()

// Step 2: Main Processing Loop
WHILE video_stream.isOpened() DO

    status, frame = video_stream.read()
    IF NOT status THEN
        BREAK // End of video feed
    END IF

    // Preprocessing & Background Subtraction
    gray_frame = ConvertToGrayscale(frame)
    blurred_frame = GaussianBlur(gray_frame, kernel=(5, 5))
    fg_mask = bg_subtractor.apply(blurred_frame)

    // Morphological operations to clean noise and join vehicle blobs
    kernel = GetStructuringElement(MORPH_RECT, size=(5, 5))
    cleaned_mask = MorphologyEx(fg_mask, MORPH_CLOSE, kernel)
    cleaned_mask = Threshold(cleaned_mask, value=200, max_val=255)

    // Object Detection via Contours
    contours = FindContours(cleaned_mask, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)
    detected_bounding_boxes = []

    FOR EACH contour IN contours DO
        IF ContourArea(contour) > 800 THEN // Minimum size filter for vehicles
            (x, y, w, h) = GetBoundingBox(contour)
            detected_bounding_boxes.append((x, y, w, h))
        END IF
    END FOR

    // Step 3: Multi-Object Tracking across Frames
```

```

        tracked_objects = tracker.update(detected_bounding_boxes)

// Step 4: Line Crossing & Counting Logic
FOR EACH (object_id, data) IN tracked_objects DO
    cx = data.centroid_x
    cy = data.centroid_y
    (x, y, w, h) = data.box

    // Annotate tracked vehicle
    DrawRectangle(frame, (x, y), (x + w, y + h), COLOR_CYAN, 2)
    DrawCircle(frame, center=(cx, cy), radius=4, COLOR_RED, fill=-1)
    DrawText(frame, "ID: " + ToString(object_id), (x, y - 5), FONT_HERSHEY, 0.5, COLOR_CYAN)

    // Line crossing evaluation with double-count prevention
    IF (cy >= LINE_Y_COORDINATE - LINE_TOLERANCE) AND
        (cy <= LINE_Y_COORDINATE + LINE_TOLERANCE) THEN

        IF object_id NOT IN counted_ids_set THEN
            total_vehicle_count = total_vehicle_count + 1
            counted_ids_set.add(object_id) // Register unique ID
        END IF
    END IF
END FOR

// Step 5: Visual Overlay & On-Screen Display (OSD)
DrawLine(frame, start=(0, LINE_Y_COORDINATE), end=(frame.width, LINE_Y_COORDINATE),
COLOR_YELLOW, 2)
DrawText(frame, "COUNTING LINE", (10, LINE_Y_COORDINATE - 10), FONT_HERSHEY, 0.5,
COLOR_YELLOW)

hud_text = "Total Vehicles Counted: " + ToString(total_vehicle_count)
DrawSolidBox(frame, (10, 10), (300, 50), COLOR_BLACK)
DrawText(frame, hud_text, (20, 35), FONT_HERSHEY, 0.7, COLOR_WHITE)

// Render Output
DisplayWindow("Traffic Monitoring System", frame)

// Efficient Loop Termination
key = WaitKey(delay = 1)
IF key == ASCII_ESC OR key == 'q' THEN
    BREAK
END IF

END WHILE

// Cleanup
video_stream.release()
DestroyAllWindows()

END Vehicle_Detection_Tracking_Counting

```